

Lucrarea de laborator 5

Sistem de gestionare a rezervărilor hoteliere. Refactorizare

Problema: Gestionarea informațiilor angajaților într-o companie

Într-un hotel, se dorește gestionarea eficientă a rezervărilor, incluzând adăugarea, vizualizarea, căutarea după ID, ștergerea și sortarea acestora. Fiecare rezervare include informații despre client, tipul camerei, numărul de nopți și prețul total al rezervării. Sistemul trebuie să permită efectuarea acestor operații într-un mod organizat și să susțină salvarea și încărcarea datelor din fișiere.

Scopul lucrării:

Lucrarea are scopul de a familiariza studenții cu utilizarea pointerilor și a structurilor de date pentru gestionarea unui sistem de rezervări într-un hotel, realizând operații de adăugare, căutare, ștergere și sortare a rezervărilor, precum și gestionarea fișierelor.

Obiective:

1. Refactorizarea unui cod scris în main, împărțindu-l în funcții separate pentru fiecare operație;
2. Utilizarea pointerilor pentru manipularea eficientă a datelor stocate în structuri dinamice;
3. Implementarea unor funcții pentru adăugarea, eliminarea și actualizarea rezervărilor;
4. Sortarea rezervărilor după diverse criterii (ex. preț total, nume client);
5. Filtrarea rezervărilor pentru a afișa doar cele premium;
6. Salvarea și citirea datelor dintr-un fișier pentru persistența informațiilor.

Cerințele lucrării:

1. Se oferă un cod inițial în care întreaga logică este implementată în funcția main. Studenții trebuie să îl refactorizeze astfel încât să folosească funcții separate;
2. Implementarea unei structuri de bază pentru rezervări, incluzând informațiile esențiale (ID, nume client, tip cameră, număr de nopți, preț total)
3. Utilizarea pointerilor;
4. Adăugarea unei funcționalități de căutare după ID-ul rezervării și afișarea detaliilor corespunzătoare.
5. Implementarea unei metode de sortare a rezervărilor, după criterii precum prețul total sau numele clientului.
6. Realizarea unei funcționalități de filtrare, permițând afișarea doar a rezervărilor în baza unui criteriu;
7. Ștergerea unei rezervări pe baza identificatorului unic;
8. Modificarea detaliilor unei rezervări existente, cum ar fi numărul de nopți sau tipul camerei;
9. Calcularea veniturilor totale generate de rezervările existente.

10. La adăugarea sau actualizarea rezervărilor, să se verifice validitatea datelor introduse (de exemplu, numărul de nopți să fie un număr pozitiv, prețul total să fie pozitiv, etc.).
11. Salvarea și preluarea datelor în fișier.

Codul inițial:

```
#include <iostream>
#include <cstring>

using namespace std;

struct Rezervare {
    int id;
    char numeClient[50];
    char tipCamera[20];
    int nopti;
    float pretTotal;
};

int main() {
    int capacitate = 10, numarRezervari = 0;
    Rezervare* rezervari = new Rezervare[capacitate];

    while (true) {
        cout << "\n1. Adaugă rezervare";
        cout << "\n2. Afișează rezervările";
        cout << "\n3. Caută rezervare după ID";
        cout << "\n4. Șterge rezervare";
        cout << "\n5. Sortează după preț";
        cout << "\n6. Salvează în fișier";
        cout << "\n7. Încarcă din fișier";
        cout << "\n0. Ieșire";
        cout << "\nAlege o opțiune: ";

        int optiune;
        cin >> optiune;

        if (optiune == 1) { // Adăugare rezervare
            if (numarRezervari >= capacitate) {
                capacitate *= 2;
                Rezervare* temp = new Rezervare[capacitate];
                for (int i = 0; i < numarRezervari; i++) {
                    temp[i] = rezervari[i];
                }
                delete[] rezervari;
                rezervari = temp;
            }

            cout << "ID: ";
            cin >> rezervari[numarRezervari].id;
            cout << "Nume client: ";
            cin.ignore();
            cin.getline(rezervari[numarRezervari].numeClient, 50);
            cout << "Tip cameră: ";
            cin.getline(rezervari[numarRezervari].tipCamera, 20);
            cout << "Număr de nopți: ";
            cin >> rezervari[numarRezervari].nopti;
            cout << "Preț total: ";
            cin >> rezervari[numarRezervari].pretTotal;

            numarRezervari++;
        }

        else if (optiune == 2) { // Afișare rezervări
            for (int i = 0; i < numarRezervari; i++) {
                cout << rezervari[i].id << " | " << rezervari[i].numeClient
                    << " | " << rezervari[i].tipCamera << " | "
            }
        }
    }
}
```

```

                << rezervari[i].nopti << " nopti | "
                << rezervari[i].pretTotal << " lei\n";
            }
        }

    else if (optiune == 3) { // Căutare după ID
        int idCautat;
        cout << "ID căutat: ";
        cin >> idCautat;
        bool gasit = false;
        for (int i = 0; i < numarRezervari; i++) {
            if (rezervari[i].id == idCautat) {
                cout << "Rezervare găsită: " << rezervari[i].numeClient << " - "
                    << rezervari[i].tipCamera << " (" << rezervari[i].nopti
                    << " nopti, " << rezervari[i].pretTotal << " lei)\n";
                gasit = true;
                break;
            }
        }
        if (!gasit) cout << "Rezervare inexistentă!\n";
    }

    else if (optiune == 4) { // Ștergere rezervare
        int idSters;
        cout << "ID rezervare de șters: ";
        cin >> idSters;
        int indexSters = -1;
        for (int i = 0; i < numarRezervari; i++) {
            if (rezervari[i].id == idSters) {
                indexSters = i;
                break;
            }
        }
        if (indexSters != -1) {
            for (int i = indexSters; i < numarRezervari - 1; i++) {
                rezervari[i] = rezervari[i + 1];
            }
            numarRezervari--;
            cout << "Rezervare ștearsă!\n";
        } else {
            cout << "Rezervare inexistentă!\n";
        }
    }

    else if (optiune == 5) { // Sortare după preț
        for (int i = 0; i < numarRezervari - 1; i++) {
            for (int j = i + 1; j < numarRezervari; j++) {
                if (rezervari[i].pretTotal > rezervari[j].pretTotal) {
                    swap(rezervari[i], rezervari[j]);
                }
            }
        }
        cout << "Sortare realizată!\n";
    }

    else if (optiune == 6) { // Salvare în fișier
        FILE* fisier = fopen("rezervari.txt", "w");
        if (fisier) {
            for (int i = 0; i < numarRezervari; i++) {
                fprintf(fisier, "%d,%s,%s,%d,%.2f\n", rezervari[i].id,
rezervari[i].numeClient,
rezervari[i].tipCamera, rezervari[i].nopti,
rezervari[i].pretTotal);
            }
            fclose(fisier);
            cout << "Rezervările au fost salvate în fișier.\n";
        } else {
            cout << "Eroare la deschiderea fișierului!\n";
        }
    }
}

```

```

        else if (optiune == 7) { // Încărcare din fișier
            FILE* fisier = fopen("rezervari.txt", "r");
            if (fisier) {
                numarRezervari = 0;
                while (fscanf(fisier, "%d,%49[^\n],%19[^\n],%d,%f\n",
&rezervari[numarRezervari].id,
                                rezervari[numarRezervari].numeClient,
rezervari[numarRezervari].tipCamera,
                                rezervari[numarRezervari].nopti,
&rezervari[numarRezervari].pretTotal) == 5) {
                    numarRezervari++;
                    if (numarRezervari >= capacitate) {
                        capacitate *= 2;
                        Rezervare* temp = new Rezervare[capacitate];
                        for (int i = 0; i < numarRezervari; i++) {
                            temp[i] = rezervari[i];
                        }
                        delete[] rezervari;
                        rezervari = temp;
                    }
                }
                fclose(fisier);
                cout << "Rezervările au fost încărcate din fișier.\n";
            } else {
                cout << "Eroare la deschiderea fișierului!\n";
            }
        }

        else if (optiune == 0) { // Ieșire
            break;
        }

        else {
            cout << "Opțiune invalidă!\n";
        }
    }

    delete[] rezervari;
    return 0;
}

```